

Bloque 09 - SQL, NoSQL y almacenamiento

Propósito

Una empresa no almacena los datos para que un analista pueda hacer una consulta bonita: los almacena primero para cobrar, servir una pantalla o registrar una acción. Este bloque enseña a distinguir esos objetivos y a consultar sin alterar el significado del dato.

Seguiremos a **Lumen Market**, una app de comercio. Sus clientes hacen pedidos, cada pedido tiene líneas, puede tener un pago y deja eventos de producto. Con ese caso aprenderás SQL sobre una base local reproducible y decidirás cuándo un documento MongoDB o una tabla DynamoDB es apropiada. No se presupone que sepas qué es una tabla, un archivo o una clave: cada uno se introduce en contexto.

Resultados observables

Al terminar podrás:

- declarar el grano, la clave primaria y la cardinalidad antes de escribir SQL;
- ejecutar y explicar una consulta con filtros, agrupaciones, CASE, HAVING, JOIN, CTE y ventanas;
- detectar duplicación, ausencias y definiciones de funnel incompatibles;
- modelar un pedido como documento y justificar *embedding* frente a referencias;
- partir de patrones de acceso para proponer claves y un GSI de DynamoDB, sin confundirlo con un warehouse;
- explicar por qué los datos OLTP de una aplicación suelen transformarse antes de un análisis OLAP.

Caso y laboratorio

El laboratorio [Lumen Market SQL](#) crea una base SQLite temporal con DDL y datos semilla. No instala nada: ejecuta `python notebooks/practicas/09-lumen-market-sql.py`. SQLite usa SQL estándar en gran parte; en la lección se indican las diferencias cuando DuckDB o un warehouse ofrecen otra sintaxis.

Lecciones

1. [Modelo relacional, grano y ERD](#)
2. [SQL básico: seleccionar, filtrar y resumir](#)

3. JOIN, cardinalidad y anti-joins
4. CTE, ventanas, fechas, nulos y funnel
5. MongoDB: documentos, pipeline e índices
6. DynamoDB: patrones de acceso, claves y GSI
7. OLTP, OLAP, warehouse, lakehouse y AI

Práctica evaluable

Resuelve el [caso de ingresos y conversión](#) sin mirar la [solución razonada](#). La competencia se evalúa por el grano, los controles y la interpretación; no por memorizar palabras clave.

01. Modelo relacional, grano y ERD

Resultado y prerequisites

Al acabar podrás mirar una pregunta de negocio, decir qué representa una fila y dibujar las relaciones que una consulta debe respetar. No necesitas haber usado una base de datos; conviene haber leído el bloque 01 sobre filas, columnas y claves.

Del recibo a las tablas

Imagina que Lumen Market recibe este pedido: Ana compra dos cafés y un té. El programa debe saber quién compró, cuándo, qué artículos había y cuánto se pagó. Podría guardar todo como texto, pero después sería muy difícil responder «¿cuántos clientes distintos compraron?». Una **base de datos** es un sistema que guarda información estructurada y permite buscarla con reglas.

Una **tabla** organiza hechos del mismo tipo: columnas para atributos y filas para casos. En un modelo **relacional**, tablas distintas se conectan por identificadores. Una **clave primaria (PK)** es el valor que identifica de forma única una fila dentro de su tabla; una **clave foránea (FK)** guarda el identificador de otra tabla para expresar una relación.

La pregunta que evita la mayor parte de los errores es el **grano**: «¿qué representa exactamente una fila?». En pedidos, una fila es un pedido; en `lineas_pedido`, una fila es un artículo dentro de un pedido. No son intercambiables.

El modelo de Lumen Market

La pregunta «¿qué conecta clientes, pedidos, líneas, pagos y eventos?» se responde con este diagrama entidad-relación simplificado:



Interpretación: un cliente puede no tener pedidos o tener muchos; un pedido tiene varias líneas; el ejercicio impone como simplificación un único pago por pedido. `UNIQUE(pedido_id)` en pagos expresa esa última regla. En un sistema real podría existir reintento, reembolso o pago dividido; entonces el grano de pagos y la relación cambiarían.

Por qué el grano cambia una métrica

Supón que P100 tiene dos líneas de 8 y 4 EUR. Esta consulta calcula ingresos por pedido correctamente:

```
SELECT pedido_id, SUM(cantidad * precio_unitario) AS importe
FROM lineas_pedido
GROUP BY pedido_id;
```

Pero si unes ese resultado con una tabla que contiene dos filas por pedido y vuelves a sumar, podrías obtener 24,80 en vez de 12,40. SQL no sabe cuál era tu definición de ingreso: ejecutará una combinación legal aunque la métrica sea falsa. Por ello se comprueban PK, FK, unicidad y recuentos antes y después de cada unión.

DDL: convertir el contrato en reglas

DDL (Data Definition Language) es SQL para declarar la estructura. El laboratorio contiene el DDL completo; este fragmento muestra las reglas importantes:

```
CREATE TABLE pedidos (
  pedido_id TEXT PRIMARY KEY,
  cliente_id TEXT NOT NULL REFERENCES clientes(cliente_id),
  creado_en TEXT NOT NULL,
  estado TEXT NOT NULL CHECK (estado IN ('pagado', 'cancelado'))
);

CREATE TABLE lineas_pedido (
  linea_id INTEGER PRIMARY KEY,
  pedido_id TEXT NOT NULL REFERENCES pedidos(pedido_id),
  producto TEXT NOT NULL,
  cantidad INTEGER NOT NULL CHECK (cantidad > 0),
  precio_unitario REAL NOT NULL CHECK (precio_unitario >= 0)
```

);

NOT NULL no permite ausencia, CHECK impone una regla y REFERENCES exige que el pedido referido exista cuando la base activa integridad referencial. Estas protecciones reducen errores, pero no sustituyen una definición de negocio: que un pedido esté pagado no demuestra por sí solo que el ingreso deba reconocerse ese día.

Error frecuente y comprobación

Error: Llamar «clientes activos» a COUNT(*) de eventos. Ese conteo mide acciones, no personas; una misma persona puede hacer diez acciones. Decide primero la unidad: cliente, sesión, pedido o línea.

Preguntas de comprobación:

1. ¿Cuál es el grano de `lineas_pedido` y por qué no es el mismo que `pedidos`?
2. ¿Qué regla protege `UNIQUE(pedido_id)` en `pagos`?
3. Si quieres contar compradores únicos, ¿qué identificador necesitas deduplicar?

Resumen y siguiente paso

Antes de consultar, formula el grano y las cardinalidades. En la siguiente lección convertirás una pregunta concreta en SELECT, filtros y agregaciones, sin perder esa disciplina.

02. SQL básico: seleccionar, filtrar y resumir

Resultado y prerequisites

Escribirás una consulta que responda «¿qué pedidos pagados tuvo Lumen por canal?» y explicarás qué filas descarta. Debes saber el grano de las tablas del caso.

SQL responde una pregunta, no adivina la métrica

SQL (Structured Query Language) es un lenguaje declarativo: indicas el conjunto de datos que quieres y el motor decide cómo obtenerlo. Una consulta no es una fórmula mágica: su resultado depende de tabla, filtros, periodo, grano y medida elegidos.

Antes de teclear, escribe un contrato mínimo:

Pedidos confirmados (estado = 'pagado') creados entre 2026-07-01 inclusive y 2026-07-08 exclusive, una fila final por canal; medida: número de pedido_id.

La fecha final exclusiva evita ambigüedad con horas. En datos con zona horaria, registra y compara instantes con zona explícita; no arregles el problema convirtiendo una fecha a texto de forma arbitraria.

SELECT, FROM, WHERE, ORDER BY

```
SELECT pedido_id, cliente_id, creado_en, canal
FROM pedidos
WHERE estado = 'pagado'
      AND creado_en >= '2026-07-01'
      AND creado_en < '2026-07-08'
ORDER BY creado_en, pedido_id;
```

FROM elige las filas de partida; WHERE conserva solo las que cumplen una condición; SELECT muestra o calcula columnas; ORDER BY ordena el resultado de presentación. Es útil leer el SQL en este orden lógico, aunque se escriba empezando por SELECT.

Contraejemplo: WHERE creado_en = '2026-07-01' puede no encontrar un instante 2026-07-01T10:15:00Z. Una fecha de calendario y un instante no son necesariamente el mismo tipo de dato.

GROUP BY: cambiar el grano de salida

Una agregación reduce varias filas a un resumen. Al agrupar por canal, el resultado deja de estar a grano pedido y pasa a grano canal:

```
SELECT
  canal,
  COUNT(*) AS filas,
  COUNT(DISTINCT pedido_id) AS pedidos_unicos
FROM pedidos
WHERE estado = 'pagado'
GROUP BY canal
ORDER BY pedidos_unicos DESC;
```

En esta tabla pedido_id es PK, por lo que ambos conteos coinciden. Escribir ambos durante una validación hace visible el supuesto. Tras un JOIN con líneas, COUNT(*) ya contaría líneas combinadas; COUNT(DISTINCT p.pedido_id) seguiría contando pedidos.

La diferencia entre filtrar antes y después del resumen es fundamental:

```
-- WHERE decide qué pedidos participan.
SELECT canal, COUNT(*) AS pedidos
FROM pedidos
WHERE estado = 'pagado'
GROUP BY canal
HAVING COUNT(*) >= 2;
```

HAVING filtra **grupos** ya formados. No uses HAVING estado = 'pagado': es una condición de fila y pertenece a WHERE.

CASE: clasificar sin borrar el dato original

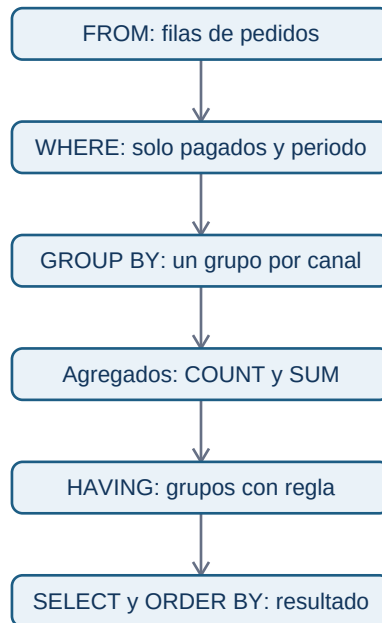
CASE crea una categoría calculada. Lumen quiere separar pedidos de importe alto y bajo, pero la regla debe ser visible y revisable:

```
SELECT
  CASE WHEN importe_total >= 20 THEN 'alto' ELSE 'habitual' END AS tramo,
  COUNT(*) AS pedidos,
  ROUND(SUM(importe_total), 2) AS ingresos
FROM (
  SELECT p.pedido_id, SUM(l.cantidad * l.precio_unitario) AS importe_total
  FROM pedidos p
  JOIN lineas_pedido l ON l.pedido_id = p.pedido_id
  WHERE p.estado = 'pagado'
  GROUP BY p.pedido_id
) AS pedido_importe
GROUP BY tramo
ORDER BY ingresos DESC;
```

Primero se calcula un importe **por pedido**; después se clasifica. Clasificar directamente cada línea respondería otra pregunta. Si importe_total fuese NULL, el CASE tomaría ELSE; decide si esa ausencia significa cero, error o dato pendiente antes de etiquetarla.

Recorrido de una consulta

La pregunta «¿qué sucede antes de que aparezca el resultado?» se resume así:



El orden enseña por qué una condición de fila no se comporta igual que una condición de grupo. Un motor puede optimizar internamente el plan, pero el significado lógico debe mantenerse.

Comprobación y práctica

Antes de confiar en un total, ejecuta una consulta de control: cuenta los pedidos por estado, inspecciona ejemplos de borde y conserva el periodo en el título de la salida. La consulta del laboratorio imprime estas comprobaciones.

Preguntas:

1. ¿Qué diferencia hay entre `COUNT( *)` y `COUNT(DISTINCT pedido_id)` después de unir líneas?
2. ¿Por qué `HAVING` no reemplaza a `WHERE`?
3. ¿Qué grano tiene el resultado de la subconsulta `pedido_importe`?

Continúa con [joins y cardinalidad](#), donde el grano puede romperse sin que SQL produzca error.

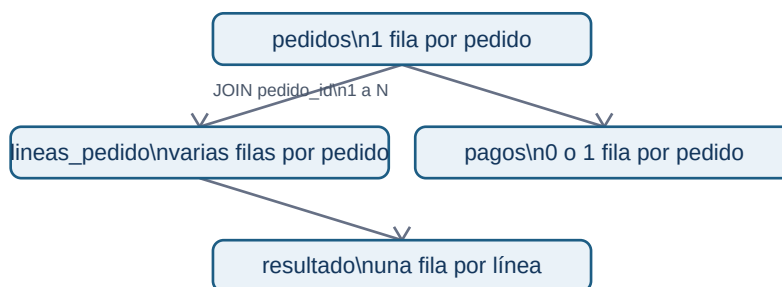
03. JOIN, cardinalidad y anti-joins

Resultado y prerequisites

Combinarás tablas de Lumen sin inflar ingresos y localizarás registros sin correspondencia. Debes conocer PK, FK y grano.

Unir es emparejar, no «añadir columnas»

Un JOIN combina una fila izquierda con las filas derechas que satisfacen una condición. La **cardinalidad** describe cuántas coincidencias puede haber: 1:1, 1:N, N:1 o N:M. Declárala antes de ejecutar la consulta; es un supuesto de negocio verificable.



El resultado de pedidos JOIN lineas_pedido está a grano **línea**, no pedido. Por eso SUM(p.importe_de_pedido) tras ese join repetiría el importe de cada pedido tantas veces como líneas tenga.

INNER JOIN y LEFT JOIN

```
-- Solo pedidos cuyos clientes existen: útil para medir integridad, pero puede ocultar fallos.  
SELECT p.pedido_id, c.pais  
FROM pedidos AS p  
INNER JOIN clientes AS c ON c.cliente_id = p.cliente_id;
```

```
-- Todos los pedidos, también si falta cliente: útil para investigar el fallo.  
SELECT p.pedido_id, c.pais  
FROM pedidos AS p  
LEFT JOIN clientes AS c ON c.cliente_id = p.cliente_id;
```

INNER JOIN conserva coincidencias de ambos lados. LEFT JOIN conserva todas las filas de la izquierda y pone NULL en columnas derechas cuando no hay coincidencia. Elegir uno cambia la población medida; no es una preferencia estética.

Validar una unión

En una relación N:1 desde pedidos hacia clientes, el número de pedidos no debe aumentar. Convierte ese razonamiento en controles:

```
-- Las claves de cliente deben ser únicas antes de la unión.
SELECT cliente_id, COUNT(*) AS n
FROM clientes
GROUP BY cliente_id
HAVING COUNT(*) > 1;

-- Recuento de filas antes y después: debe coincidir para N:1.
SELECT COUNT(*) AS pedidos_antes FROM pedidos;
SELECT COUNT(*) AS pedidos_despues
FROM pedidos p LEFT JOIN clientes c ON c.cliente_id = p.cliente_id;
```

Un resultado vacío en el primer control y recuentos iguales son evidencia de que este aspecto del join es seguro. No prueban que país esté actualizado ni que la definición de cliente sea correcta.

Agregar antes de unir cuando hace falta

La regla práctica es: si necesitas una métrica por pedido a partir de líneas, agrega las líneas a pedido **antes** de combinarlas con otra relación N.

```
WITH importe_por_pedido AS (
  SELECT pedido_id, SUM(cantidad * precio_unitario) AS importe
  FROM lineas_pedido
  GROUP BY pedido_id
)
SELECT c.pais, COUNT(*) AS pedidos, ROUND(SUM(i.importe), 2) AS ingresos
FROM pedidos p
JOIN importe_por_pedido i ON i.pedido_id = p.pedido_id
JOIN clientes c ON c.cliente_id = p.cliente_id
WHERE p.estado = 'pagado'
GROUP BY c.pais;
```

La CTE deja claro que `importe_por_pedido` tiene una fila por pedido. En la siguiente lección se estudia esta construcción con más detalle.

Anti-join: encontrar lo que falta

Un **anti-join** devuelve filas de un lado que no encuentran pareja en el otro. Es imprescindible para calidad, conciliación y funnels. La forma más clara suele ser `NOT EXISTS`:

```
-- Pedidos pagados que no tienen pago liquidado: anomalía a investigar.
SELECT p.pedido_id, p.creado_en
FROM pedidos p
WHERE p.estado = 'pagado'
AND NOT EXISTS (
  SELECT 1 FROM pagos g
  WHERE g.pedido_id = p.pedido_id AND g.estado = 'liquidado'
);
```

También puede expresarse con `LEFT JOIN ... WHERE g.pago_id IS NULL`. Evita `NOT IN` si la subconsulta puede contener `NULL`: la lógica ternaria de SQL puede producir un resultado inesperado.

Error habitual y resumen

Error: usar `SELECT DISTINCT` al final para «arreglar» duplicados. Puede ocultar una relación mal modelada y descartar filas legítimas. Primero descubre qué tabla multiplicó las filas y a qué grano debe quedar el resultado.

Preguntas: ¿qué cardinalidad tiene `clientes` hacia pedidos? ¿Qué control ejecutarías antes de sumar dinero tras un join? ¿Qué pregunta responde el anti-join de pagos?

En la siguiente lección construirás consultas en pasos, compararás filas con ventanas y validarás un funnel temporal.

04. CTE, ventanas, fechas, nulos y funnel

Resultado y prerrequisitos

Construirás una consulta analítica por pasos, compararás cada pedido con el anterior de su cliente y medirás un funnel sin contar eventos como personas. Requiere saber agrupar y unir.

CTE: una consulta que se puede revisar

Una **CTE** (common table expression) da nombre a un resultado intermedio mediante `WITH`. No es automáticamente más rápida: su valor principal para el analista es separar transformaciones con un grano claro.

```
WITH pedidos_pagados AS (  
    SELECT pedido_id, cliente_id, creado_en  
    FROM pedidos  
    WHERE estado = 'pagado'  
)  
, importe_por_pedido AS (  
    SELECT pedido_id, SUM(cantidad * precio_unitario) AS importe  
    FROM lineas_pedido  
    GROUP BY pedido_id  
)  
SELECT p.cliente_id, p.pedido_id, i.importe  
FROM pedidos_pagados p  
JOIN importe_por_pedido i USING (pedido_id);
```

Cada CTE permite comprobar una cosa: `pedidos_pagados` tiene un pedido por fila; `importe_por_pedido` también. Si el resultado es extraño, inspecciona cada CTE por separado antes de añadir más SQL.

Ventanas: calcular sin perder filas

GROUP BY reduce filas; una **función de ventana** calcula sobre un grupo relacionado y conserva el detalle. En Lumen, ROW_NUMBER() enumera pedidos de cada cliente y LAG() trae el dato anterior según un orden explícito:

```
WITH importe AS (
  SELECT pedido_id, SUM(cantidad * precio_unitario) AS total
  FROM lineas_pedido GROUP BY pedido_id
)
SELECT p.cliente_id, p.pedido_id, p.creado_en, i.total,
       ROW_NUMBER() OVER (
         PARTITION BY p.cliente_id ORDER BY p.creado_en, p.pedido_id
       ) AS numero_pedido,
       LAG(i.total) OVER (
         PARTITION BY p.cliente_id ORDER BY p.creado_en, p.pedido_id
       ) AS total_anterior
FROM pedidos p JOIN importe i USING (pedido_id)
WHERE p.estado = 'pagado';
```

PARTITION BY reinicia la ventana por cliente; ORDER BY define qué significa «anterior». Sin orden completo, dos filas con la misma hora pueden hacer el resultado no determinista. El primer pedido tiene NULL en total_anterior: significa que no existe uno anterior, no importe cero.

Fechas y nulos: ausencia no equivale a cero

Una fecha puede ser una fecha de negocio, un instante UTC o la hora local del usuario. Declara cuál usas. Para intervalos, usa extremos como [inicio, fin) y deja la zona horaria en el contrato.

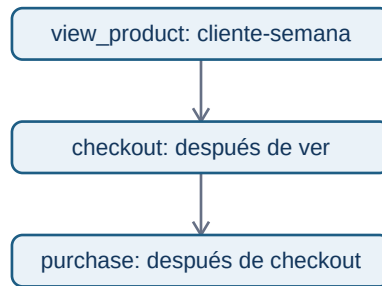
NULL significa «desconocido, no aplicable o no registrado», según la fuente. COALESCE(valor, 0) solo es correcto si una regla de negocio dice que la ausencia representa cero:

```
SELECT p.pedido_id, COALESCE(g.importe, 0) AS importe_cobrado
FROM pedidos p LEFT JOIN pagos g USING (pedido_id);
```

En una conciliación financiera, sustituir un pago ausente por cero puede esconder un fallo. Es mejor exponer una columna de estado y tratar la ausencia explícitamente.

Funnel que conserva su definición

Un **funnel** mide cuántas personas o entidades pasan por pasos ordenados. No es «contar eventos de cada nombre»: una persona puede disparar checkout_started repetidamente. El contrato de Lumen es: cliente que hizo view_product y, después, checkout_started y purchase durante la misma semana UTC. Se cuenta una vez por cliente y semana.



Una implementación pedagógica usa la primera hora de cada paso y condiciones de orden:

```
WITH por_cliente_semana AS (  
  SELECT cliente_id, substr(ocurrido_en, 1, 10) AS dia,  
    MIN(CASE WHEN evento = 'view_product' THEN ocurrido_en END) AS vista,  
    MIN(CASE WHEN evento = 'checkout_started' THEN ocurrido_en END) AS checkout,  
    MIN(CASE WHEN evento = 'purchase' THEN ocurrido_en END) AS compra  
  FROM eventos GROUP BY cliente_id, substr(ocurrido_en, 1, 10)  
)  
SELECT COUNT(*) AS vistas,  
  SUM(checkout >= vista) AS checkout_despues_de_vista,  
  SUM(compra >= checkout AND checkout >= vista) AS compras_validas  
FROM por_cliente_semana WHERE vista IS NOT NULL;
```

Para producción, define semana con una función de calendario del motor, usa una tabla de fechas y decide qué hacer con eventos sin `cliente_id`, duplicados y zonas horarias. El laboratorio muestra ambos conteos para detectar cambios de definición.

Resumen y controles

Una consulta mantenible declara pasos, granos, orden y significado de ausencias. Comprueba que las etapas del funnel disminuyen, inspecciona IDs de ejemplo y compara el conteo de eventos con el de clientes únicos. Sigue con [MongoDB](#): cambiar de modelo no elimina estos contratos.

05. MongoDB: documentos, pipeline e índices

Resultado y prerequisites

Podrás representar un pedido como JSON, decidir entre incrustar y referenciar, y leer un pipeline de agregación sin confundir flexibilidad con falta de reglas. No hace falta instalar MongoDB.

Un documento antes que la jerga

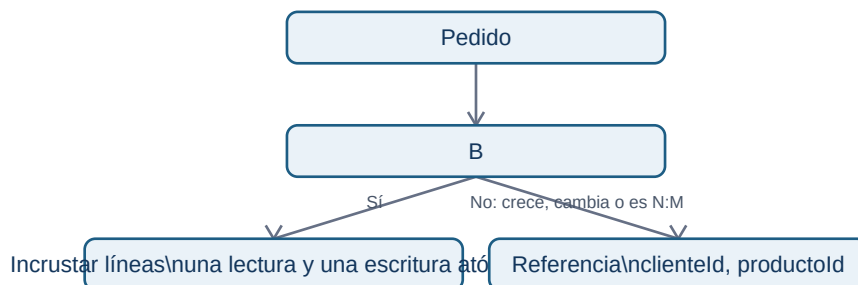
Un **documento** es una pieza de información con campos y puede contener objetos o listas. **JSON** es una forma de escribir esa estructura en texto. Para una pantalla de detalle de pedido, resulta natural leer el pedido y sus líneas juntos:

```
{
  "_id": "P100",
  "clienteId": "C001",
  "creadoEn": "2026-07-01T10:15:00Z",
  "estado": "pagado",
  "lineas": [
    {"producto": "cafe", "cantidad": 2, "precioUnitario": 4.00},
    {"producto": "te", "cantidad": 1, "precioUnitario": 4.40}
  ]
}
```

MongoDB agrupa documentos en **colecciones**. Un documento no requiere que todos tengan exactamente las mismas propiedades, pero una aplicación profesional conserva un contrato: identificadores, tipos, versión de esquema, campos obligatorios y semántica de importes.

Incrustar o referenciar

La pregunta «¿cuándo conviene guardar juntos los datos relacionados?» tiene dos respuestas posibles:



Embedding guarda datos relacionados dentro de un documento y puede servirlos en una sola lectura; es útil para líneas de un pedido cerrado. Las [guías oficiales de MongoDB sobre embedding](#) recalcan esas ventajas. Usa **referencias** si duplicar sería costoso, los datos cambian con frecuencia, hay relaciones muchos-a-muchos o un arreglo puede crecer sin control; la [documentación oficial](#) enumera estos casos.

No copies el precio actual del catálogo para rehacer un pedido histórico. En una línea de pedido normalmente se conserva un *snapshot* del precio vendido; eso es una decisión de negocio documentada, no una propiedad automática de MongoDB.

Pipeline de agregación

Un **pipeline** procesa documentos por etapas. Este resume ingresos de pedidos pagados; primero filtra, después expande líneas, luego calcula y agrupa:

```
db.pedidos.aggregate([
  {$match: {estado: "pagado", creadoEn: {$gte: ISODate("2026-07-01")}}},
  {$unwind: "$lineas"},
  {$group: {
    _id: "$clienteId",
    ingresos: {$sum: {$multiply: ["$lineas.cantidad", "$lineas.precioUnitario"]}},
    pedidos: {$addToSet: "$_id"}
  }},
  {$project: {ingresos: 1, pedidos: {$size: "$pedidos"}}},
  {$sort: {ingresos: -1}}
]);
```

\$unwind cambia el grano de pedido a línea, igual que un join 1:N en SQL. Por eso los pedidos se cuentan mediante conjunto de IDs, no con el número de documentos tras la expansión. Añade y prueba índices en los campos de filtro y orden de tus patrones reales; un índice acelera algunas lecturas a cambio de espacio y coste de escritura. Usa `explain()` y datos representativos antes de afirmar que una consulta es rápida.

AI y controles

Un asistente que genera un pipeline desde lenguaje natural produce un borrador. Verifica colección, periodo, tipo de fecha, grano tras \$unwind, permisos, índices, coste y ejemplos manuales. Ningún texto generado por AI conoce por defecto qué significa «ingreso» en Lumen.

Preguntas: ¿por qué las líneas de un pedido pueden incrustarse? ¿qué cambia \$unwind? ¿cuándo una referencia es más segura?

Sigue con [DynamoDB](#), donde primero se diseñan los accesos, no las entidades.

06. DynamoDB: patrones de acceso, claves y GSI

Resultado y prerequisites

Propondrás un diseño de DynamoDB para dos lecturas conocidas de Lumen y explicarás qué pregunta analítica no resuelve. Requiere comprender grano y claves; no requiere una cuenta AWS.

Empezar por la pregunta que ejecuta la aplicación

DynamoDB es una base NoSQL gestionada de clave-valor/documento. En vez de comenzar por un diagrama de entidades ideal, se enumeran **patrones de acceso**: qué se lee o escribe, con qué clave,

en qué orden, cuántas veces y con qué latencia.

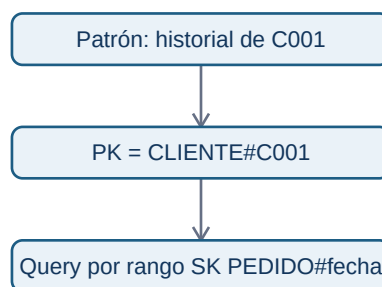
Para Lumen:

Los tres primeros son candidatas a GetItem o Query. El último necesita explorar muchas particiones y agregación: es trabajo de warehouse, no una razón para hacer Scan periódico sobre la tabla operacional.

Clave primaria compuesta

La clave primaria puede ser simple o compuesta. Una clave compuesta tiene **PK** (partition key) y **SK** (sort key). Las filas con la misma PK forman una colección de ítems y se ordenan por SK. AWS documenta que una PK debe distribuir carga y que la SK permite rangos y relaciones uno-a-muchos: [fundamentos de modelado](#) y [buenas prácticas de sort key](#).

Un diseño posible de tabla única para el historial del cliente es:



La pregunta agregada global de la tabla anterior no debe resolverse con un Scan periódico: se extrae a OLAP. La SK debe ordenarse para los rangos que realmente se consultan. Una PK con valor muy repetido, como estado=pagado, puede concentrar tráfico; AWS recomienda distribuir actividad de forma uniforme y analizar volumen por clave ([partition keys](#)).

GSI: otro camino de consulta, no una búsqueda gratis

Un **Global Secondary Index (GSI)** reorganiza ítems con otra clave para soportar un patrón adicional. Para la cola operativa podrías escribir GSI1PK = ESTADO#pagado#2026-07-01 y GSI1SK = creado_en#pedido_id; entonces consultas un día y estado sin recorrer toda la tabla. Un GSI tiene su propio esquema de clave y capacidad; la [documentación oficial](#) explica sus atributos proyectados y límites de consulta.

Antes de añadirlo pregunta: ¿qué ítems lo tienen?, ¿cuánto escriben?, ¿su PK distribuye carga?, ¿qué atributos necesita la lectura? Un índice de baja cardinalidad puede convertirse en cuello de botella y aumentar el coste de escritura.

Límites y relación con analítica

DynamoDB no ofrece joins arbitrarios ni está pensado para descubrir después cualquier agregación histórica. Desnormalizar para una lectura conocida puede ser correcto en OLTP; para métricas reproducibles exporta cambios a una capa analítica, conserva historia y define transformaciones. No declares que una tabla «es el warehouse» porque guarde muchos datos.

Preguntas: ¿qué patrón justifica un GSI? ¿qué haría peligrosa una PK ESTADO#pagado? ¿por qué ingresos trimestrales por país no es una Query natural?

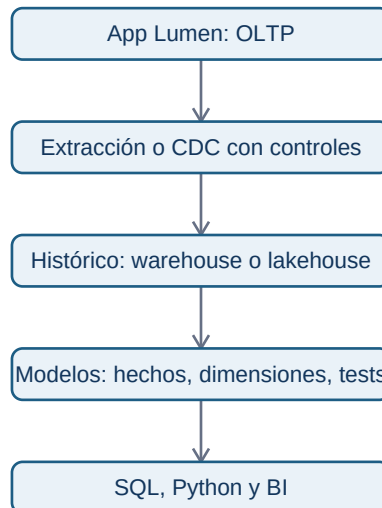
07. OLTP, OLAP, warehouse, lakehouse y AI

Resultado y prerequisites

Distinguirás la base que sostiene una operación de la capa que permite analizarla, y revisarás una consulta asistida por AI mediante controles reproducibles.

Dos trabajos distintos

OLTP (procesamiento transaccional en línea) prioriza registrar una operación correcta y rápida: crear pedido, cobrar o mostrar perfil. Sus tablas o ítems suelen estar optimizados para accesos de la aplicación. **OLAP** (procesamiento analítico en línea) prioriza leer muchas observaciones, combinar historia y resumir para responder preguntas. Un análisis de margen trimestral por cohortes no debe competir con la pantalla de pago.



El diagrama no afirma que toda empresa tenga las mismas herramientas. Expresa una separación de responsabilidades: la copia analítica recibe datos, los transforma de forma documentada y sirve decisiones sin alterar el registro operacional.

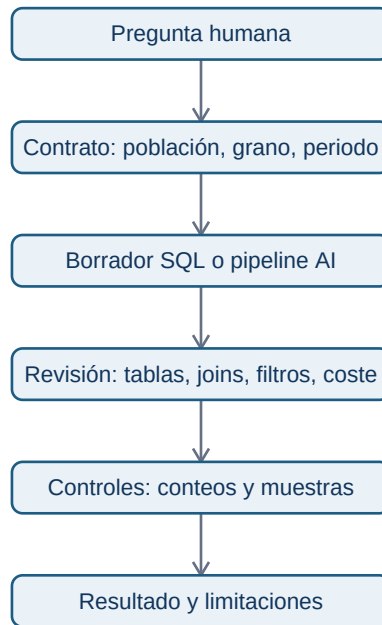
Warehouse y lakehouse sin promesas vacías

Un **warehouse** organiza datos limpiados y modelados para análisis; suele exponer tablas de hechos (eventos medibles, como pedido) y dimensiones (contexto, como cliente o calendario). Un **lakehouse** combina almacenamiento de archivos de distintos tipos con capacidades de tabla y consulta analítica. Los nombres comerciales cambian; para el analista importan linaje, calidad, permisos, coste, granularidad y refresco.

Ejemplo: `fact_pedidos` puede tener una fila por pedido pagado, con `dim_fecha` y `dim_cliente` vinculadas. No copies la tabla operacional sin pensar: decide la hora de corte, reembolsos, deduplicación, zona horaria y cómo se corrigen datos tardíos. Eso convierte datos en una fuente defendible.

Consultas generadas por AI: borrador verificable

Una herramienta puede transformar «ingresos de julio por país» en SQL o MongoDB. Acelera sintaxis, pero no decide qué significa ingreso ni conoce los permisos. Revisa siempre este flujo:



Un checklist mínimo:

1. ¿Usa la tabla certificada y campos con la definición vigente?
2. ¿El grano final coincide con la pregunta? ¿un join multiplica filas?
3. ¿La ventana de fecha, zona y estado de pedidos están explícitos?
4. ¿NULL, reembolsos y datos tardíos tienen tratamiento declarado?
5. ¿La ejecución respeta permisos y no expone datos personales innecesarios?
6. ¿Hay conteos intermedios y una muestra manual para contrastar?

Cierre del bloque

SQL, MongoDB y DynamoDB son herramientas para contratos diferentes. La habilidad profesional es conservar el significado de cada observación desde el evento operacional hasta la métrica. Ejecuta el laboratorio, resuelve el caso evaluable y guarda las consultas con su pregunta, fuente, fecha de ejecución y controles.